

WasmRef-Isabelle

or how to formally verify a not-slow interpreter with not-insane amount of effort

Maja Trela¹

¹affiliation note: worked on this at University of Cambridge, now employed at Jane Street

RESEARCH-ARTICLE

OPEN ACCESS

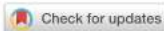


WasmRef-Isabelle: A Verified Monadic Interpreter and Industrial Fuzzing Oracle for WebAssembly

Authors: [Conrad Watt](#), [Maja Trela](#), [Peter Lammich](#), [Florian Märkl](#) [Authors Info & Claims](#)

Proceedings of the ACM on Programming Languages, Volume 7, Issue PLDI • Article No.: 110, pp 100–123 • <https://doi.org/10.1145/3591224>

Published: 06 June 2023 [Publication History](#)



WebAssembly (Wasm)



- ▶ Executable bytecode format
- ▶ Precise small-step semantics
- ▶ Strongly-typed

Wasmtime

- ▶ Cross-platform WebAssembly runtime
- ▶ Focused on correctness and security

Features

- **Fast.** Wasmtime is built on the optimizing [Cranelift](#) code generator to quickly generate high-quality machine code either at runtime or ahead-of-time. Wasmtime is optimized for efficient instantiation, low-overhead calls between the embedder and wasm, and scalability of concurrent instances.
- **Secure.** Wasmtime's development is strongly focused on correctness and security. Building on top of Rust's runtime safety guarantees, each Wasmtime feature goes through careful review and consideration via an [RFC process](#). Once features are designed and implemented, they undergo 24/7 fuzzing donated by [Google's OSS Fuzz](#). As features stabilize they become part of a [release](#), and when things go wrong we have a well-defined [security policy](#) in place to quickly mitigate and patch any issues. We follow best practices for defense-in-depth and integrate protections and mitigations for issues like Spectre. Finally, we're working to push the state-of-the-art by collaborating with academic researchers to formally verify critical parts of Wasmtime and Cranelift.

Wasmtime fuzzing

- ▶ Fuzzing: comparing Wasmtime with a reference implementation on random input
- ▶ Use the official reference interpreter?

README.md

WebAssembly Reference Interpreter

This repository implements an interpreter for WebAssembly. It is written for clarity and simplicity, *not* speed. It is intended as a playground for trying out ideas and a device for nailing down their exact semantics. For that purpose, the code is written in a fairly declarative, "speccky" way.

Reference interpreter problem

- ▶ Evaluation contexts are inefficient
- ▶ Execution time scales quadratically



alexorichon commented on Aug 13, 2021

Member

I... don't really know how we want to handle this. It appears that the spec interpreter is showing extremely slow behavior when processing nested blocks. For example this module (redude from a fuzz case):

```
(module
  (%no
    global, get 0
    102, 002
    if
      unreachable
    end
    global, get 0
    137, const 1
    102, sub
    global, set 0

    block block block block block block
    block block block block block block
    block block block block block block
    call 0
    end end end end end end
    end end end end end end
    end end end end end end
  )
  (global {out 102} {102, const 1000})
  (return " (true 0)"))
```

takes many dozens of seconds to run in the interpreter. If the `block ... end` is all removed though it finishes near instantaneously.

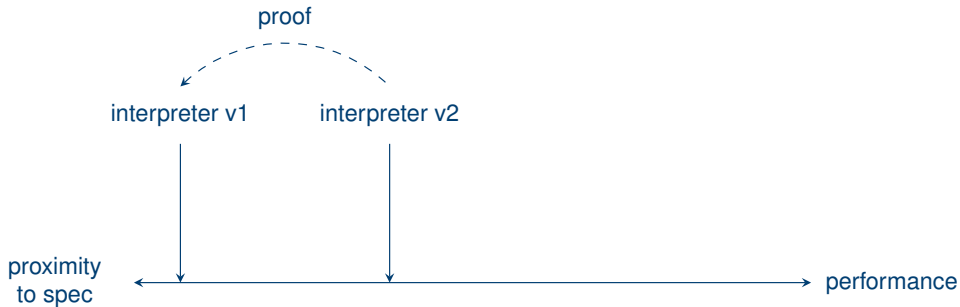
Interpreter tradeoffs



Can't move towards one without giving up on the other!

What if...?

Could we do something like this?



WasmRef-Isabelle

This is exactly WasmRef-Isabelle:

- ▶ Verified with respect to WebAssembly semantics
- ▶ Fast enough for use in fuzzing; adopted by Wasmtime team
 - ▶ Running in their CI infrastructure!

WasmCert-Isabelle

Building on WasmCert-Isabelle:



- ▶ Mechanization of WebAssembly semantics in Isabelle/HOL
- ▶ Notably, found type safety bugs before WebAssembly release
- ▶ Interpreter written in Isabelle/HOL, extracted to OCaml

What are we verifying?

- ▶ We prove *soundness*:

for every initial configuration,
if the interpreter terminates without crashing,
the result is consistent with the given specification

- ▶ Note: says nothing about it not crashing

```
theorem intermediate_interpreter_sound:  
  assumes "interpreter start_config = (new_store, RValue values)"  
  shows "accepted_by_semantics start_config new_store values"
```

Interpreters

Pure interpreter:

- ▶ Sound wrt. spec
- ▶ Tweaked WasmCert-Isabelle interpreter
- ▶ No mutable state, uses lists ($O(n)$ memory access)

Monadic interpreter:

- ▶ Sound wrt. pure interpreter
- ▶ Mutable arrays ($O(1)$ memory access)
- ▶ Needs state monad

Comparison

```
definition load
  :: "mem  $\Rightarrow$  nat  $\Rightarrow$  off  $\Rightarrow$  nat  $\Rightarrow$  bytes option"
  where
    "load m n off l =
      (if (mem_length m  $\geq$  (n+off+l))
        then Some (read_bytes m (n+off) l)
        else None)"
```

```
definition load_bytes_m_v
  :: "mem_m  $\Rightarrow$  nat  $\Rightarrow$  off  $\Rightarrow$  nat  $\Rightarrow$  (bytes option) Heap"
  where
    "load_bytes_m_v m n off l =
      do {
        m_len  $\leftarrow$  len_byte_array (fst m);
        let ind = n+off;
        (if (m_len  $\geq$  (ind+l)) then do {
          bs  $\leftarrow$  load_uint8_list (fst m) ind l;
          return (Some bs) }
        else
          return None)
      }"
```

'a Heap:

```
text <Monadic heap actions either produce values
and transform the heap, or fail>
datatype 'a Heap = Heap "heap  $\Rightarrow$  ('a  $\times$  heap) option"
```

Proof

```
lemma load_bytes_triple:
  shows "<mem m assn m m>
        load_bytes m v m m n off l
        <λr. ↑(r = load m n off l) * mem m assn m m>"
  unfolding load_bytes m v def load def mem m assn def
  by (sep_auto simp add: read_bytes_def mem_rep_read_bytes_def
      mem_length_def mem_rep_length.rep_eq split: prod.splits)
```

- ▶ Relate two implementations with Hoare triples
- ▶ Separation logic: Sepref (modified)
- ▶ Automation: sep_auto, Sledgehammer

Sledgehammer!

```
proof (prove)
goal (4 subgoals):
1.  $\bigwedge x1\ x2\ x1a\ a\ b.$ 
    $m\ m = (x1a,\ x2) \implies$ 
    $m = (x1,\ x2) \implies$ 
    $n + off + l \leq mem\_length\ (x1,\ x2) \implies$ 
    $n + off + l \leq length\ (Rep\_mem\_rep\ x1) \implies$ 
    $0 < l \implies$ 
    $(a,\ b) \vdash x1a \mapsto_{ba} Rep\_mem\_rep\ x1 \implies$ 
    $take\ l\ (drop\ (n + off)\ (Rep\_mem\_rep\ x1)) = read\_bytes\ (x1,\ x2)\ (n + off)\ l$ 
```

Provers:

☐ Isar proofs

☒ Try methods 

Apply

Locate

Proof found...

"verit": Try this: `apply (metis mem_rep_read_bytes.rep_eq prod.sel(1) read_bytes_def)` (80 ms)

"z3": Try this: `apply (metis mem_rep_read_bytes.rep_eq prod.sel(1) read_bytes_def)` (83 ms)

"vampire": Try this: `apply (metis mem_rep_read_bytes.rep_eq prod.sel(1) read_bytes_def)` (104 ms)

Not always so smooth

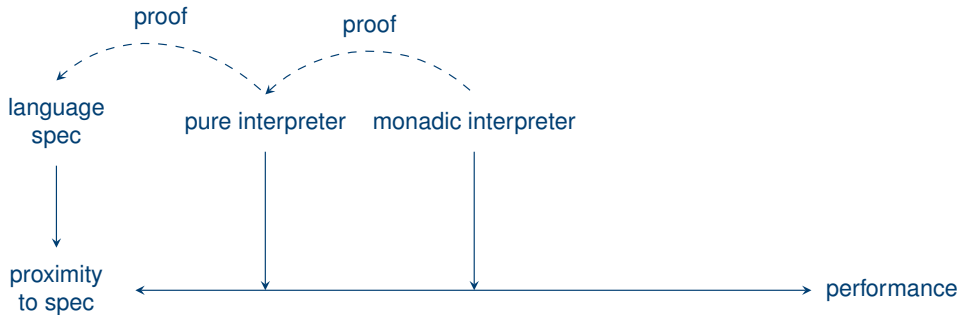
```
lemma store_vec_triple:
  shows "<mems_n, assn ms, ms_n * inst_n, assn (f_inst f) f_inst2>
  app s f v s store vec m sv off ms m f_inst2 v s
  <λr. let (ms', v s', res) = app s f v s store vec sv off ms f v s in
  [(r = (v s', res))
  * mems_n, assn ms' ms_m * inst_n, assn (f_inst f) f_inst2> "
  unfolding app s f v s store vec m def app s f v s store vec def
  inst n assn def list assn conv idx mems n assn def
  apply (sep_auto split: v.num.splits v.splits v_vec.splits prod.splits)
  apply (extract_reinsert_list_idx "inst.mems (f_inst f) ! 0")
  apply (sep_auto)
  apply (extract_list_idx "inst.mems (f_inst f) ! 0")
  apply (sep_auto heap: store_uint8_list_triple)
  apply (sep_auto)
  apply (sep_auto simp: smem ind def split: list.splits prod.splits)
  apply (sep_auto simp: store def Let def)
  apply (sep_auto simp: store_def Let def)
  apply (rule listI_assn_reinsert_upd, frame_inference, simp, simp)
  apply (sep_auto simp: store def Let def bytes takefill def)
  apply (sep_auto simp: smem ind def store def split: list.splits)
  apply (sep_auto)+
done
```

Issues

- ▶ Higher-order functions
- ▶ Shared mutable states
- ▶ “UX” problems

Proof summary

Two-step *refinement proof*



Interpreter performance

Good worst-case performance!

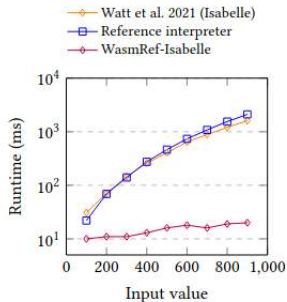


Fig. 12. Graphing execution times for the function $f(0) \triangleq 0; f(n+1) \triangleq f(n) + n^5$ (note the log scale for runtime).

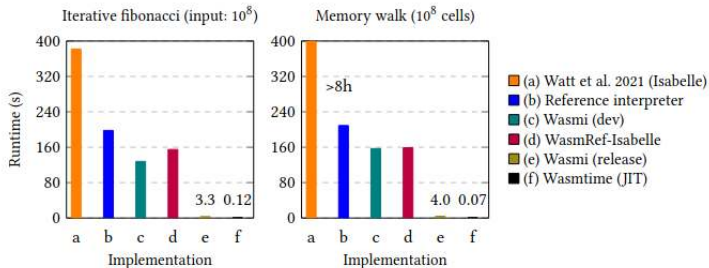


Fig. 13. Iterative fibonacci and memory walk benchmarks.

Interpreter performance (cont.)

No longer the bottleneck in fuzzing!

oracle	total tests	slowest test (secs)
Reference Interpreter	1900091	48
WasmRef-Isabelle	2740908	< 1
Wasmi (release)	2793736	< 1

Conclusions

- ▶ Monadic soundness proof done in a few months as a 4th year project
- ▶ How to verify a not-slow interpreter with a not-insane amount of effort?
 - ▶ Refinement proof + decent automation!
- ▶ Only possible due to good tools
 - ▶ Neel Krishnaswami, “The Golden Age of PL Research”
- ▶ As costs fall we should expect more projects!

Thank you for listening!

Email: maja@majatrelda.com

References:

- ▶ Paper dl.acm.org/doi/abs/10.1145/3591224
- ▶ Wasmtime fuzzing
bytecodealliance.org/articles/security-and-correctness-in-wasmtime
- ▶ Proof repo github.com/WasmCert/WasmCert-Isabelle/tree/new_interp_2021-1
- ▶ The Golden Age of PL Research
semantic-domain.blogspot.com/2022/09/the-golden-age-of-pl-research.html
- ▶ Personal (still incomplete) website: majatrelda.com